

ALTEN SHOP — Attendus & choix techniques

Ce que la consigne demande, comment on compte le réaliser, et pourquoi — avec les justifications à présenter au recruteur.

Cahier des charges

Plan de réalisation

Justifications techniques

Bilan de réalisation (mise à jour finale)

✓ Projet livré en full-stack — 100 % du sujet + bonus, déployé en ligne

Ce document décrivait le *plan* et les choix envisagés. Voici ce qui a été **effectivement réalisé** :

- **Back** : Spring Boot 4 (Java 21), CRUD produits, JWT (BCrypt), contrôle admin (`admin@admin.com`), panier & wishlist par utilisateur, base H2, Swagger.
- **Front** : affichage produits enrichi, panier (badge + page + persistance), page Contact validée, pagination + filtrage, connexion/inscription, interceptor JWT, guard, boutons de gestion réservés à l'admin.
- **Tests** : JUnit/Mockito/MockMvc côté back, **Jest** côté front (écrits au fur et à mesure).
- **Industrialisation** : Docker + déploiement sur **Render** (front + back).

Écarts assumés par rapport au plan initial ci-dessous

- **Tests front** : **Jest** retenu au lieu de Karma (déprécié).
- **Hébergement** : **Render** pour le front ET le back (au lieu de Vercel), pour rester sur une seule plateforme.
- **Base** : **H2** conservée (pas de Supabase/PostgreSQL) — suffisant pour la démo ; bascule possible sans changer le code grâce à JPA.
- **Panier** : géré côté front (Signals + localStorage) ; l'API panier du back existe et est testée (branchement possible en évolution).

La suite de ce document conserve l'analyse des exigences et la justification des choix techniques (utile pour la soutenance).

0. Comment lire ce document

Chaque exigence du `README.md` est reprise une à une. Pour chacune :

- un rappel de **ce qui est demandé** ;
- un statut : **FAIT** **PARTIEL** **À FAIRE** ;
- **comment on le fait** + le **pourquoi** (l'argument technique).

Décision structurante recommandée

Se positionner en **full-stack** : compléter le front Angular fourni *et* créer le back-end. C'est l'option qui valorise le plus de compétences et qui rend l'app réellement fonctionnelle (persistance + sécurité). La consigne impose alors une **stratégie de test de l'API**.

1. Front-end — Partie 1 : Shop

1.1 — Afficher toutes les informations pertinentes d'un produit dans la liste PARTIEL

Existant : la liste n'affiche que *catégorie* + *nom*.

Approche : enrichir le `p-card` du `p-dataView` avec image (`assets` ou URL), prix formaté (`| currency: 'EUR'`), note (`p-rating`), et un badge de stock coloré à partir de `inventoryStatus` (vert/orange/rouge via `p-tag`).

Pourquoi : on réutilise les composants PrimeNG déjà importés (cohérence visuelle, accessibilité), et le *currency pipe* Angular gère le formatage localisé (locale fr déjà enregistrée).

1.2 — Ajouter un produit au panier depuis la liste À FAIRE

Approche : créer un `CartService` (`providedIn: 'root'`) sur le même patron que `ProductsService` : un `signal<CartItem[]>` privé + `asReadOnly()` , et des méthodes `addToCart(product)` , `removeFromCart(id)` , `updateQuantity(id, q)` . Un bouton « Ajouter au panier » sur chaque `p-card` .

Pourquoi : rester homogène avec l'architecture existante (Signals + service singleton). Un `computed()` dérivera le total d'articles et le prix total sans logique dupliquée.

1.3 — Supprimer un produit du panier À FAIRE

Approche : méthode `removeFromCart(id)` du `CartService` (mise à jour du signal), bouton « Supprimer » dans la vue panier.

1.4 — Badge indiquant la quantité de produits dans le panier À FAIRE

Approche : brancher l'icône panier déjà présente dans la toolbar (`app.component.html`) sur un `p-badge` alimenté par un `computed(() => total d'articles)` du `CartService` .

Pourquoi : l'icône existe déjà → le badge se met à jour automatiquement grâce à la réactivité des signals, sans code impératif.

1.5 — Visualiser la liste des produits du panier À FAIRE

Approche : nouvelle route `/cart` + `CartComponent` affichant les lignes (image, nom, prix unitaire, quantité, sous-total) et le total général. Accès via clic sur l'icône panier (ajouter un `routerLink`) et/ou une entrée de menu.

2. Front-end — Partie 2 : Page Contact

2.1 à 2.5 — Menu + page + formulaire validé À FAIRE

Demandé : un point de menu « Contact », une page avec formulaire (email + message), validations (les deux obligatoires, message < 300 caractères), et un message de succès « Demande de contact envoyée avec succès » à l'envoi.

Approche :

- Ajouter l'entrée *Contact* dans `panel-menu.component.ts` (`routerLink:['/contact']`).
- Créer `ContactComponent` (feature) + route `/contact`.
- Formulaire avec **Reactive Forms** : `Validators.required` + `Validators.email` sur l'email, `Validators.required` + `Validators.maxLength(300)` sur le message. Bouton « Envoyer » désactivé tant que le formulaire est invalide.
- À la soumission : afficher un **toast** via le `MessageService` PrimeNG (déjà fourni globalement) avec le texte exact demandé.

Pourquoi Reactive Forms plutôt que template-driven ? Validation déclarative, testable, et compteur de caractères facile à brancher sur la valeur du contrôle. (Le formulaire produit existant utilise template-driven : on peut assumer ce choix différent en expliquant que Reactive Forms est mieux adapté à des validations multiples.)

3. Front-end — Bonus

Bonus	Approche	Pourquoi
Pagination et/ou filtrage des produits	Activer la pagination native du <code>p-dataView</code> (<code>[paginator]="true"</code> <code>[rows]="..."</code>) ; filtrage par nom/catégorie via un <code>signal</code> de critère + <code>computed</code> de liste filtrée.	Le <code>p-dataView</code> est déjà en place : pagination « gratuite ». Le filtrage en <code>computed</code> reste réactif et sans effet de bord.
Ajuster la quantité depuis la liste et le panier	<code>p-inputNumber</code> avec boutons +/- lié à <code>updateQuantity()</code> du <code>CartService</code> .	Composant déjà importé dans le projet (formulaire produit) → réutilisation directe.

4. Back-end — Technologie imposée : Java / Spring Boot

La consigne autorise Node/Express, Java/Spring Boot, C#/.NET ou PHP/Symfony (API Platform interdite). **Choix retenu : Java / Spring Boot** (exigence du recruteur).

Stack Java retenue

Brique	Outil	Rôle
Framework	Spring Boot 3.x (Java 17+)	Socle de l'application, auto-configuration, serveur Tomcat embarqué
API REST	Spring Web (<code>spring-boot-starter-web</code>)	Contrôleurs <code>@RestController</code> , mapping des routes
Persistance	Spring Data JPA / Hibernate	Entités <code>@Entity</code> + <code>Repository</code> (CRUD sans SQL manuel)
Base de données	H2 (dev) → PostgreSQL (prod)	H2 en mémoire pour démarrer vite ; PostgreSQL pour une base « sérieuse »
Sécurité	Spring Security + JWT (<code>jjwt</code>)	Filtre d'authentification, hachage, protection des routes
Validation	Bean Validation (<code>jakarta.validation</code>)	Annotations <code>@NotNull</code> , <code>@Email</code> , <code>@Size</code> sur les DTO
Documentation	springdoc-openapi (Swagger UI)	Doc interactive générée automatiquement
Tests	JUnit 5 + Mockito + MockMvc	Tests unitaires et d'intégration des endpoints
Build	Maven (ou Gradle)	Gestion des dépendances et build (<code>mvn spring-boot:run</code>)

Arguments à présenter à l'oral

- **Standard d'entreprise** : Spring Boot est la référence Java pour les API REST, écosystème mature et très demandé en ESN.
- **Architecture en couches claire** : `Controller` → `Service` → `Repository` → `Entity` , séparation des responsabilités, testable.
- **Sécurité robuste** : Spring Security est éprouvé pour le JWT et le contrôle d'accès, plutôt que de réécrire l'authentification à la main.

Organisation dans le dépôt

Le projet Spring Boot vivra dans le dossier `back/` (aujourd'hui vide). Structure type :

`back/src/main/java/com/alten/shop/` avec les sous-packages `controller`, `service`, `repository`, `entity`, `dto`, `security`, `config` ; et `back/src/main/resources/application.yml` pour la config (datasource, secret JWT).

Persistance des données

Option	Quand la choisir
H2 (base SQL en mémoire/fichier)	<i>Recommandé pour démarrer.</i> Zéro installation, intégré à Spring Boot, console web pour inspecter les données. Parfait pour une démo et les tests.
PostgreSQL	Pour une base « production » crédible. Nécessite un serveur (ou Docker). On peut basculer H2→PostgreSQL en changeant uniquement <code>application.yml</code> grâce à JPA.
Fichier JSON	Autorisé par la consigne, mais on perd tout l'intérêt de JPA (relations users↔cart↔products, requêtes). À éviter avec Spring.

Pourquoi JPA/Hibernate : on décrit les tables comme des classes Java annotées (`@Entity`) et Spring génère le CRUD et le schéma automatiquement — on se concentre sur le métier, pas sur le SQL.

5. Back-end — Partie 1 : API de gestion des produits

CRUD Produits À FAIRE

Demandé : gérer les produits (modèle `Product` imposé) en base.

Endpoints REST à exposer :

```
GET    /products      liste de tous les produits
GET    /products/:id    détail d'un produit
POST   /products      création           (admin only)
PATCH /products/:id  mise à jour       (admin only)
DELETE /products/:id  suppression       (admin only)
```

Détails : `createdAt` / `updatedAt` gérés côté serveur (callbacks JPA `@PrePersist` / `@PreUpdate`), validation des champs entrants (DTO + Bean Validation : `@NotBlank`, `@Positive` ...), codes HTTP corrects (201 création, 404 introuvable via `@ResponseStatus` / exception handler, 400 payload invalide).

Mise en œuvre Spring : entité `Product` annotée `@Entity`, interface `ProductRepository extends JpaRepository<Product, Long>` (CRUD fourni), `ProductService` pour le métier, `ProductController` pour les routes REST.

Cohérence front : le service Angular appelle déjà `/api/products` en GET/POST/PATCH/DELETE. Il suffira d'aligner le préfixe (proxy Angular `/api` → back) pour que le front bascule du mock au vrai back sans refonte.

6. Back-end — Partie 2 : Authentification & droits

6.1 — Création de compte & connexion JWT À FAIRE

Demandé : deux routes —

```
POST /account { username, firstname, email, password } → crée l'utilisateur
POST /token   { email, password } → renvoie un JWT
```

Approche (Spring Security) :

- **Hachage du mot de passe** avec `BCryptPasswordEncoder` (bean Spring, jamais de mot de passe en clair en base).
- `/account` : vérifie l'unicité de l'email, encode le mot de passe, enregistre l'utilisateur (entité `User` + `UserRepository`).
- `/token` : retrouve l'utilisateur par email, compare le mot de passe (`passwordEncoder.matches`), puis génère un **JWT** signé avec la librairie `jjwt` — claims contenant l' `email`, un secret défini dans `application.yml`, et une date d'expiration.

Pourquoi JWT : sans état (stateless), le serveur n'a pas à stocker de session ; le front renvoie le token dans l'en-tête `Authorization: Bearer ...`, validé par un filtre Spring Security à chaque requête.

6.2 — Protéger l'API (connexion obligatoire) À FAIRE

Approche : un **filtre Spring Security** (`OncePerRequestFilter` ajouté à la `SecurityFilterChain`) qui extrait et valide le JWT sur les routes protégées, et place l'utilisateur dans le `SecurityContext`. Côté Angular, un **HTTP interceptor** ajoute automatiquement le header `Bearer` et gère le 401 (redirection login).

Pourquoi un interceptor : centralise l'ajout du token → aucun appel HTTP n'a à s'en soucier.

6.3 — Seul `admin@admin.com` peut créer/modifier/supprimer un produit À FAIRE

Demandé : solution « simple et générique », sans gestion de rôles complète.

Approche : après validation du JWT, on vérifie que l' `email` du token vaut `admin@admin.com`. Deux façons en Spring : soit un test explicite dans le service, soit donner l'autorité `ROLE_ADMIN` à cet email et protéger les routes de mutation avec `@PreAuthorize("hasRole('ADMIN')")` (ou une règle dans la `SecurityFilterChain`). Appliqué uniquement sur POST/PATCH/DELETE `/products` ; les GET restent accessibles à tout utilisateur connecté.

Pourquoi : répond exactement à la consigne (« pas de RBAC ») tout en restant propre et centralisé — une seule comparaison, réutilisable.

6.4 & 6.5 — Panier & liste d'envies côté serveur À FAIRE

Demandé : chaque utilisateur peut gérer un panier et une liste d'envies (wishlist) contenant des produits.

Endpoints :

```
GET    /cart                panier de l'utilisateur courant
POST   /cart                { productId, quantity }
DELETE /cart/:productId
GET    /wishlist
POST   /wishlist { productId }
DELETE /wishlist/:productId
```

Modèle JPA : relations `User` 1–N `CartItem` N–1 `Product` (`@OneToMany` / `@ManyToOne`) et `User` N–N `Product` pour la wishlist (`@ManyToMany`). L'utilisateur est déduit du JWT via le `SecurityContext` (pas d' `userId` dans l'URL → on ne peut pas lire le panier d'un autre).

Sécurité : rattacher panier/wishlist à l'identité du token évite l'accès horizontal aux données d'autrui (bon point à souligner à l'oral).

7. Back-end — Bonus : documentation/tests d'API

Option	Approche	Pourquoi
Swagger	Dépendance <code>springdoc-openapi-starter-webmvc-ui</code> : génère l'UI Swagger automatiquement à partir des contrôleurs et DTO (annotations <code>@Operation</code> optionnelles).	Documentation vivante + testable dans le navigateur (<code>/swagger-ui.html</code>), quasi gratuite avec Spring.
Collection Postman	Exporter une collection avec les requêtes (account, token, products, cart, wishlist) + variable d'environnement pour le token.	Démontre les parcours bout-en-bout et se partage facilement au recruteur.

8. Choix techniques transverses (à savoir justifier)

Décision	Choix retenu	Justification courte
Langage / framework back	Java + Spring Boot 3	Standard d'entreprise, écosystème mature, exigence du recruteur
Persistance	Spring Data JPA + H2/PostgreSQL	CRUD et schéma générés, relations gérées, bascule de base facile
Mots de passe	BCryptPasswordEncoder (hash + salt)	Jamais de mot de passe en clair ; standard éprouvé
Authentification	Spring Security + JWT (jjwt) stateless	Pas de session serveur, scalable, simple côté SPA
Contrôle admin	Vérif. email du token (ROLE_ADMIN / @PreAuthorize)	« Simple et générique » comme demandé, centralisé
État front	Angular Signals + services	Réactif, natif, sans boilerplate NgRx pour ce périmètre
Validation front	Reactive Forms	Validations multiples déclaratives et testables (contact)
Token côté front	HTTP Interceptor	Ajout automatique du Bearer + gestion centralisée du 401
Persistance	SQLite + ORM	Vraie base, zéro install, relations et migrations
Doc API	Swagger + Postman	Couvre le bonus, démontre la qualité

9. Stratégie de test (exigée si full-stack)

- **Tests unitaires back** (JUnit 5 + Mockito) : services produits/auth, hachage, génération/validation du JWT, logique du contrôle admin.
- **Tests d'intégration back** (`@SpringBootTest` + `MockMvc`) : appels réels sur les endpoints (création de compte → token → accès protégé → CRUD produit en tant qu'admin vs utilisateur simple), base H2 de test.
- **Collection Postman** avec assertions (statut HTTP, présence du token) pour la démonstration manuelle.
- **Front** (optionnel mais valorisant) : tests des services `CartService` / `ProductsService` (Jasmine/Karma déjà configurés) et validation du formulaire contact.

10. Ordre de développement proposé

1. **Front rapide d'abord** (gains visibles immédiats, sans dépendre du back) : enrichir l'affichage produit → `CartService` + ajout/suppression + badge → page panier → page Contact.
2. **Back-end** : init Spring Boot (Spring Initializr) + base H2 + entité/CRUD Produits (JPA).
3. **Auth** : `/account`, `/token`, filtre JWT, contrôle admin (Spring Security).
4. **Panier & wishlist serveur** rattachés à l'utilisateur.
5. **Branchement front↔back** : interceptor JWT + proxy `/api`, retirer le fallback silencieux des mutations.
6. **Tests + Swagger/Postman** + bonus (pagination, filtrage, quantités).

11. FAQ entretien — questions probables & réponses

Question probable	Réponse à donner
Pourquoi Java / Spring Boot pour le back ?	Standard d'entreprise pour les API REST, écosystème mature, architecture en couches claire (Controller/Service/Repository), Spring Security éprouvé pour le JWT, Spring Data JPA pour la persistance, springdoc pour Swagger.
Pourquoi des Signals plutôt que NgRx ?	Périmètre modeste : NgRx ajouterait du boilerplate. Les signals donnent une réactivité fine, native, lisible, avec état encapsulé dans des services singletons.
Comment sécurisez-vous les mots de passe ?	Hachage BCrypt (BCryptPasswordEncoder) avec sel, jamais stockés/loggés en clair, comparaison via passwordEncoder.matches.
Comment marche le contrôle admin sans rôles ?	Le filtre Spring Security valide le JWT, puis on vérifie que l'email du token vaut admin@admin.com (ou via ROLE_ADMIN + @PreAuthorize) sur les routes de mutation produit. Simple, générique, centralisé.
Un utilisateur peut-il voir le panier d'un autre ?	Non : l'utilisateur est déduit du JWT, jamais d'un id en URL. Pas d'accès horizontal.
Comment le front sait-il quel produit est en stock ?	Champ inventoryStatus (INSTOCK/LOWSTOCK/OUTOFSTOCK) affiché via un tag coloré.
Que se passe-t-il si l'API tombe ?	Aujourd'hui : fallback sur le JSON local (lecture) et faux succès sur les écritures — à durcir en prod en remontant l'erreur à l'utilisateur (toast d'échec).
Comment testez-vous l'API ?	Tests unitaires (JUnit 5 + Mockito) et d'intégration (@SpringBootTest + MockMvc) sur les parcours critiques, plus une collection Postman et Swagger pour la doc/démo.
Comment branchez-vous front et back ?	Proxy Angular /api vers le serveur, interceptor HTTP pour le Bearer, alignement des endpoints déjà appelés par ProductService.

Document 2/2 — Attendus & choix techniques. À lire avec le document 1 « État des lieux ». Une fois ces deux documents assimilés, on attaque le développement dans l'ordre proposé en section 10.